# How can you securely share a secret number with someone, totally in public? Elliptic Curves

**Cite as:** Martin Paul Eve, "How can you securely share a secret number with someone, totally in public? Elliptic Curves", *eve.gd: Martin Paul Eve*, August 19, 2026, https://doi.org/10.59348/zx6pm-4nd50.

---

| Variable | What it is | Privacy | Which party |
|---|---|---|---|
| $k$ | Alice's private key. A large random number. | Private | Alice |
| $G$ | A publicly agreed base point on the curve. | Public | Alice, Bob |
| $kG$ | Alice's public key. A public point on the curve. Equivalent to $G$ self-added $k$ times. A shared public value. | Public | Alice (producer), Bob (consumer) |
| $r$ | Bob's temporary private key. A large, ephemeral random number. | Private | Bob |
| $rG$ | Ephemeral public point on the curve. Equivalent to $G$ self-added $r$ times. A shared public value. | Public | Bob (producer), Alice (consumer) |
| $k(rG) = r(kG)$ | The final, calculated, shared secret point. Bob and Alice can calculate this indepdenently of each other without transmitting it. Their respective secret numbers ($r$ and $k$), multiplied by either $rG$ or $kG$, are the same. | Private | Alice, Bob (calculated independently / individually) |

*Image: A table of elliptic curve variables (set out fully in tabular form in the post) (Martin Paul Eve)*

How do you totally securely share a secret number with someone, utterly in public, without anybody else being able to get it?

This morning, for my *Dark Web* book, I have been writing about elliptic curve cryptography and how it works, which solves this precise problem (following in the footsteps of other methods like RSA). It turns out that some mathematical curves (yes, wavy wiggly lines on a grid) have some weird mathematical properties that make this possible.

So, to use this system, we first have to pick a "curve" that has these properties. A number are known, with catchy names such as "Curve25519". These curves have a hard discrete-logarithm problem associated with them (I won't go into that here, though).

Anyway, we then take a publicly defined base point on the pre-defined curve, called $G$. Alice then chooses a very large number, $k$, to act as her private key (not shared/private). Next, this $G$ point is "added" to itself $k$ times (elliptic-curve addition is slightly different from normal addition). This gives us another point on the curve, called $kG$. This is Alice's public key (shared/public).

Then, Bob chooses his own random ephemeral secret number, $r$ (not shared/private). Bob then creates his own ephemeral public point on the curve, by adding $G$ to itself $r$ times (shared/public). So everyone knows $G$, $kG$, and $rG$. Only Alice knows $k$ and only Bob knows $r$.

The mathematical magic is that $k(rG) = r(kG) = krG$. Hence, using information only they respectively know ($k$ and $r$) combined with public points ($kG$ and $rG$), Alice and Bob can both independently calculate the same, shared secret point ($krG$). They never transmit this $krG$ value to each other; it is calculated in private, but they will have the same point.

Importantly, it is computationally infeasible to derive $r$ and $k$ from $kG$ and $rG$. This secret point, $krG$, can then be used as part of a key-derivation function to create a key (Alice and Bob will get the same key) that can be used in a traditional symmetric encryption algorithm. This is called ECDH key agreement.

| Variable | What it is | Privacy | Which party |
|---|---|---|---|
| $G$ | A publicly agreed base point on the curve. | Public | Alice, Bob |

| Variable | What it is | Privacy | Which party |
|---|---|---|---|
| $k$ | Alice's private key. A large random number. | Private | Alice |
| $kG$ | Alice's public key. A public point on the curve. Equivalent to $G$ self-added $k$ times. A shared public value. | Public | Alice (producer), Bob (consumer) |
| $r$ | Bob's temporary private key. A large, ephemeral random number. | Private | Bob |
| $rG$ | Ephemeral public point on the curve. Equivalent to $G$ self-added $r$ times. A shared public value. | Public | Bob (producer), Alice (consumer) |
| $k(rG) = r(kG)$ | The final, calculated, shared secret point. Bob and Alice can calculate this independently of each other without transmitting it. Their respective secret numbers ($r$ and $k$), multiplied by either $rG$ or $kG$, are the same. | Private | Alice, Bob (calculated independently / individually) |

As with all asymmetric cryptography, this relies on a mathematical operation that is easy to perform in one direction, but extraordinarily difficult to reverse. In this case, it is easy for Alice to calculate $kG$. But even if an attacker knows $G$ and $kG$, it is totally infeasible to calculate $k$. Going from the secret number to a public point is easy to calculate. But going back from the public point to a secret number is extraordinarily hard. That's the basic trick of elliptic curve cryptography.